

Практическая работа №3

Написание составных команд в командной оболочке `bash`

Введение

Команды сами по себе являются программами, скомпилированными в бинарные файлы, которые лежат в директории `/bin`. Вызывая команду в терминале, вы фактически запускаете отдельный процесс. Каждый запущенный процесс в системе Linux имеет связанные с ним три информационных канала:

- Стандартный поток ввода `stdin`.
- Стандартный поток вывода `stdout`.
- Стандартный поток ошибок `stderr`.

Каждый из этих каналов имеет свой файловый дескриптор. Файловый дескриптор – это индекс (целое неотрицательное число), указывающий на строку в специальной таблице, содержащую информацию об открытых процессом файлах. Стандартные потоки имеют фиксированные значения файловых дескрипторов: `STDIN` – 0, `STDOUT` – 1, `STDERR` – 2.

Кроме этого, каждый процесс связан со своим управляющим терминалом. Стандартные потоки процесса по умолчанию связаны с таким терминалом. Именно поэтому в окне нашего терминала отображается результат вывода команд. А также благодаря этой связи введенный текст при необходимости попадает на вход запущенных команд.

Эти три стандартных потока фактически выступают универсальным интерфейсом, с помощью которого мы можем взаимодействовать с процессом, связывать команды, выстраивать цепочки (пайплайны) или перенаправлять потоки в места отличные от терминала (файлы, сокет и т. д.).

Кроме ключей, аргументов и стандартного потока ввода процесс может брать входную информацию из переменных окружения. Окружение – это не что иное как строки, содержащие пары «ключ=значение». Чаще всего их используют для конфигурирования, чтобы изменять и настраивать поведение тех или иных команд. **Важно!** Далее по тексту, где указываются команды, которые необходимо вводить в терминал, знак \$

вводить не нужно. Этим знаком обозначается обычное приглашение консоли.

Полезные ссылки:

- Про параметры оболочки: <https://www.gnu.org/software/bash/manual/bash.html#Shell-Parameter-Expansion>
- Организация циклов в bash: <https://www.cyber-citi.biz/faq/linux-unix-bash-for-loop-one-line-command/>

Описание практической работы

Цель работы

Получение практических навыков по созданию составных команд в оболочке bash.

Указания к выполнению работы

Работа состоит из четырех частей. В первой части осуществляется подготовка виртуального терминала для выполнения команд, во второй части реализуется перенаправление потоков ввода и вывода, в третьей части создается пайплайн команд, в четвертой части создаются переменные командной оболочки.

Оформление отчета

Отчёт должен быть составлен в соответствии с требованиями, представленными во введении. Экспериментальную часть отчета необходимо сопроводить несколькими снимками экрана. К отчету необходимо приложить файл `record_result_lab_3`. Это текстовый файл с введенными командами. По результатам выполнения работы необходимо ответить на вопросы, насколько удобно писать сложные команды на bash, а также в чем достоинства и недостатки использования пайплайна?

Ход работы

Подготовка

- 1) Запустите виртуальную машину и дождитесь загрузки ОС.

- 2) Откройте терминал. Это можно сделать с помощью основного меню или сочетанием клавиш **Ctrl+Alt+T**. Для запуска через основное меню нажмите на Menu → System Tools → Mate Terminal.
- 3) **Запустите запись терминала. Это важно!** Именно файл с записью истории ваших команд будет основной частью отчета. Для этого введите и запустите команду:

```
$ script -a record_result_lab_3
```

- 4) Убедитесь, что команда выполнилась без ошибок. Введите нижеуказанную команду и найдите имя файла `record_result_lab_3` в ее выводе:

```
$ ls -l
```

Перенаправление

Исходный код программ `lab3.py` и `lab3_2.py` находятся в приложении Б.

- 1) Вернитесь в терминал, в котором уже идет запись (см. Подготовка).
- 2) Перейдите на рабочий стол. Для этого:

```
$ cd ./Desktop
```

в случае с русским языком нужно использовать экранирование пробела:

```
$ cd ./Рабочий\ стол/
```

- 3) Выведите список всех файлов и каталогов. Убедитесь, что в выводе есть файл `lab3.py`.

```
$ ls -l
```

- 4) Запустите генерацию вашего варианта. Вывод этой команды добавьте в отчет в виде скриншота. Для генерации:

```
$ python lab3.py
```

- 5) Снова запустите команду выше и перенаправьте стандартный поток вывод `STDOUT` в файл `фамилия_out`. Для этого используйте символ перенаправления потока `STDOUT` «>»:

```
$ python lab3.py > фамилия_out
```

- 6) Снова запустите команду выше и перенаправьте стандартный поток ошибок `STDERR` в файл `фамилия_егг`. Для этого используйте символ перенаправления потока `<>` и номером файлового дескриптора, соответствующего этому потоку. Обратите внимание, что `<>` не должен обрамляться пробелами:

```
$ python lab3.py 2>фамилия_егг
```

- 7) Откройте файлы и убедитесь, что там появился текст:

```
$ nano фамилия_out
```

```
$ nano фамилия_егг
```

- 8) Две предыдущие команды можно записать в одну строку. Созданные файлы предварительно удалите командой `rm`.

```
$ python lab3.py>фамилия_out 2>фамилия_егг
```

- 9) Проверьте, что файлы снова создались и наполнены текстом.

- 10) Как вы могли заметить, текст в этих файлах представляет собой диалог, но часть фраз находится в одном файле, а часть в другом. Давайте восстановим диалог. Для этого нам требуется перенаправить `STDOUT` и `STDIN` в один файл. Символ `&` позволяет понять командой оболочке, что `1` — это номер файлового дескриптора (`STDOUT`), а не имя файла. Для этого:

```
$ python lab3.py>фамилия_final 2>&1
```

- 11) Теперь давайте посчитаем количество слов в каждом созданном файле. Для этого воспользуемся утилитой `wc`:

```
$ wc -w < имя_файла
```

- 12) В отчет добавьте содержание полученных файлов и подсчитанное количество слов.

Пайплайн

Теперь научимся связывать стандартные потоки команд. Такая связь называется пайплайном. Вместе с этим файлом лабораторной работы для вас прикладываются файлы `the-matrix-reloaded.csv`, `variants.txt`, `lab3_2.py`.

- 1) Попробуем поискать в `the-matrix-reloaded.csv` некоторые строки по составным запросам. Для этого будем использовать утилиту `grep`. Изучите руководство пользователя по этой утилите.

```
$ man grep
```

```
$ grep pattern_for_search file.txt # Пример
использования
```

- 2) Сгенерируйте свой вариант запустив скрипт `lab3_2.py`. В той же папке должен находиться файл `variants.txt`.

```
$ python lab3_2.py
```

- 3) В терминале вы увидите следующее, пример приведен на рисунке 3.1:

- а) Чьи фразы необходимо найти
- б) Кто должен упоминаться в этих фразах
- в) Какой знак препинания должен содержаться
- г) Какой объект, явление, событие, действие упоминается

От вас потребуется найти все строки для запроса «а», затем для «а и б», затем для «а, б и в» и так далее. Далее работа будет рассматриваться на примере задания с рисунка 3.1. Вам же потребуется сделать на примере своего варианта.

```
a) The character that says a phrase is "Neo"
b) The character that is talked with/about is "Trinity"
c) Should contain punctuation ","
d) The subject that is talked about "love"
```

```
Process finished with exit code 0
```

Рисунок 3.1 – Пример результата генерации задания для пайплайна

- 4) Используйте утилиту `cat`, чтобы вывести содержимое файла в консоль. Но вывод перенаправьте с терминала на утилиту `grep`. Для этого используется символ «`|`». Найдите строки, в которых говорит «Нео» (запрос «а» из задания), для этого выполните команду:

```
$ cat the-matrix-reloaded.csv | grep Neo
```

Можно заметить, что в выводе появляются фразы, где говорит не сам Нео, а где говорят о нём. Чтобы их убрать, давайте скажем `grep`, чтобы строка начиналась с Нео. Для этого используем символ «^»:

```
$ cat the-matrix-reloaded.csv | grep ^Neo
```

- 5) Теперь добавьте к этой команде еще поиск строк, в которых Нео упоминает Trinity (запрос «б» из задания):

```
$ cat the-matrix-reloaded.csv | grep ^Neo | grep Trinity
```

Как можно увидеть из команды выше, мы можем использовать утилиту повторно и соединять результат первого использования утилиты `grep` с входом второго вызова утилиты `grep`. Это работает для всех утилит.

- 6) Теперь добавьте к этой команде еще поиск строк, в которых присутствует запятая (запрос «в» из задания).

```
$ cat the-matrix-reloaded.csv | grep ^Neo | grep Trinity  
| grep ,
```

- 7) Теперь добавьте к этой команде еще поиск строк, в которых используется слово «love»:

```
$ cat the-matrix-reloaded.csv | grep ^Neo | grep Trinity  
| grep , | grep love
```

- 8) В конце посчитаем количество полученных строк. Для этого воспользуемся утилитой `wc` (известной нам по предыдущей лабораторной работе).

```
$ cat the-matrix-reloaded.csv | grep ^Neo | grep Trinity  
| grep , | grep love | wc -l
```

- 9) А теперь давайте выведем количество строк без использования `wc`. Вы же прочитали `man` для `grep`? Найдите там необходимый ключ.

Переменные

Создание переменных может пригодиться при составлении сложных команд в терминале. Чтобы создать переменную достаточно сделать следующее:

```
$ MY_VAR=3 # создали переменную равную 3
```

```
$ echo $MY_VAR
```

Обратите внимание, что знак «=» не обрамляется пробельными символами. Далее по тексту \$ в начале строки означает приглашение к вводу и не набирается, однако **\$ в середине команды** имеет значение «раскрытие значения переменной» и должен набираться в терминале.

- 1) Возьмите персонажа из предыдущего задания по пайплайну (в примере это «Neo»). Создадим переменную с именем актера.

```
$ name='Keanu Reeves'
```

- 2) Прочитайте мануал для команды `sed`. Это утилита является потоковым редактором текста.

```
$ man sed
```

- 3) Выведите все фразы заданного персонажа так, чтобы вначале вывелось «Text for Keanu Reeves». Для этого напишем следующую команду:

```
cat the-matrix-reloaded.csv | grep ^Neo | sed "1i\ --  
Text for $name "
```

- 4) Чтобы не листать вывод, выведем только первые 5 строк:

```
$ cat the-matrix-reloaded.csv | grep ^Neo | sed "1i\  
Text for $name" | head -5
```

- 5) Теперь заменим во всех фразах имя персонажа на имя актера и запишем в файл.

```
$ cat the-matrix-reloaded.csv | grep ^Neo | sed  
"s/Neo,/$name:/" > keanu_text
```

- 6) Теперь пронумеруем строки в файле, для этого воспользуемся циклом `while`. Для этого напишем следующую команду:

```
counter=1; while read line; do echo $counter $line;  
((counter++)); done < keanu_text
```

- 7) Рассмотрим команду подробнее. Вначале создается переменная счетчика

```
counter=1
```

- 8) Затем создается шапка цикла `while`. Цикл `while` имеет следующий синтаксис:

```
while [ condition ]; do <commands>; done
```

или

```
while <control-command>; do <COMMANDS>; done
```

- 9) В шапке цикла `while` используем команду `read`. Команда `read` считывает одну строку из стандартного потока ввода.

```
while read line;
```

- 10) Затем переходим к телу цикла. В теле цикла командой `echo` выводим значение переменных `$counter` `$line`.

```
echo $counter $line
```

- 11) Затем в теле цикла инкрементируем счетчик. Это арифметическое выражение. Для подстановки результата арифметического выражения используются двойные круглые скобки:

```
((counter++))
```

- 12) Тело цикла закрывается ключевым словом `done`.

- 13) На вход всей этой сложной конструкции подаем вывод из файла `keanu_text` с помощью конструкции:

```
(ваша длинная команда с циклом) <keanu_text
```

- 14) Завершите запись команд, набрав в терминале:

```
$ exit
```

Контрольные вопросы

- 1) Перечислите потоки ввода и вывода, ассоциированные с любым процессом в операционной системе Linux.
- 2) Что такое пайплайн?
- 3) Как осуществляется перенаправление потоков ввода/вывода в командной оболочке `bash`?
- 4) Для чего используется утилита `grep`?

Приложение Б

Исходные коды программ для практической работы №3

Файл lab3.py

```
import sys
import csv
import random

CHUNK_MAX = 20
CHUNK_MIN = 10
LINE_NUM = 769

chunk_size = random.randrange(CHUNK_MIN, CHUNK_MAX)
slice_start = random.randrange(1, LINE_NUM-chunk_size)
slice_end = slice_start + chunk_size

out = [sys.stdout, sys.stderr]
with open('the-matrix-reloaded.csv', newline='') as csvfile:
    dialog_lines = list(csv.reader(csvfile))
    chosen_lines = dialog_lines[slice_start:slice_end]
    for line in chosen_lines:
        random_stream = random.choice(out)
        print(f'{line[0]}: {line[1]}', file=random_stream,
flush=True)
```

Файл lab3_2.py

```
import random

with open('variants.txt', newline='') as f:
    lines = f.readlines()
    sentence = random.choice(lines).split(' ')
    sentence = [word.strip() for word in sentence]
    saying_char = sentence[0]
    about_char = sentence[1]
    subject = sentence[2]
    punc = sentence[3]
    print(f'a) The character that says a phrase is "{saying_char}"')
```

```
print(f'\tb) The character that is talked with/about is
"{about_char}"')
print(f'\t\tc) Should contain punctuation "{punc}"')
print(f'\t\td) The subject that is talked about "{sub-
ject}"')
```